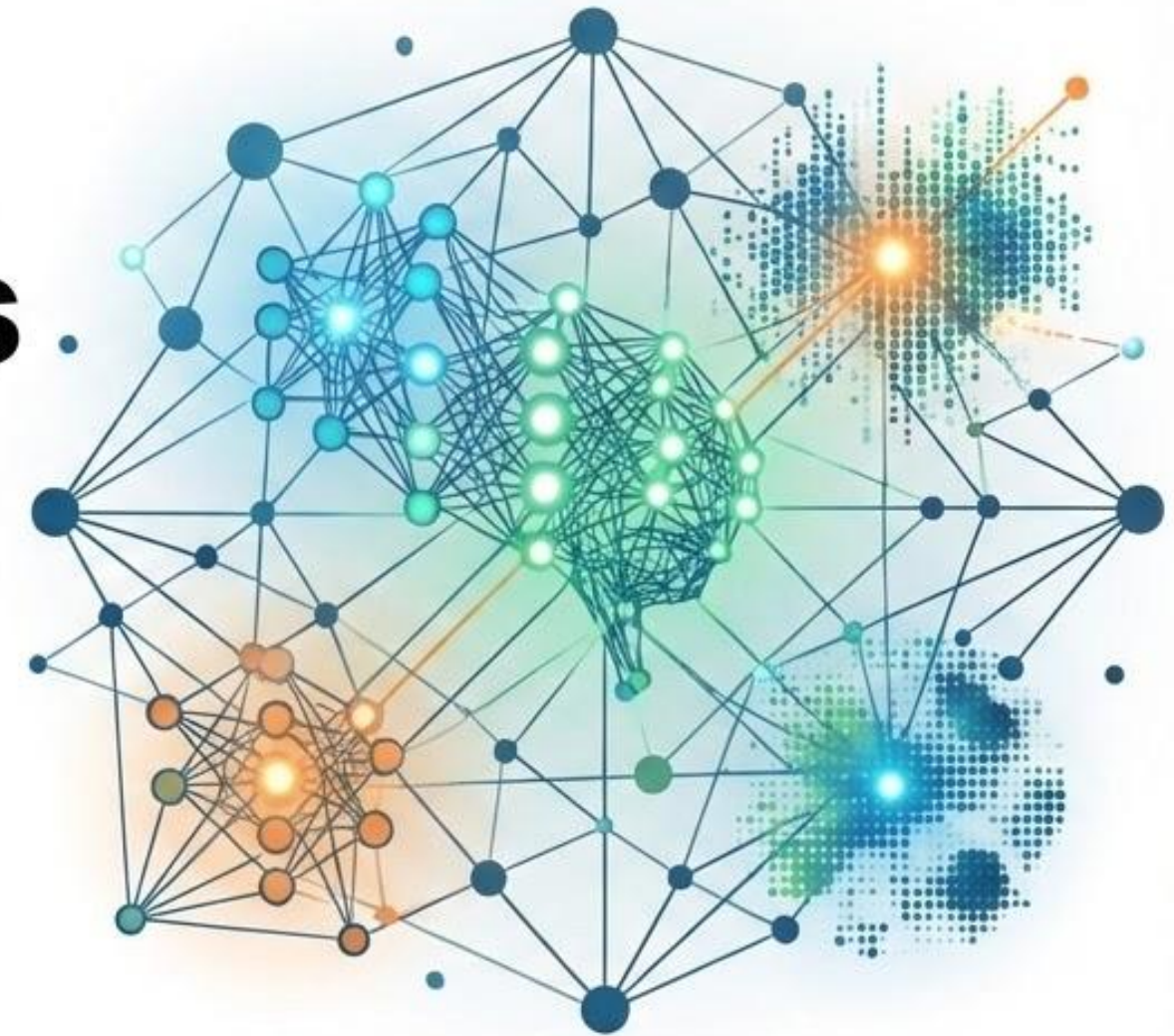


Diplomatura en python para la ciencia de datos y machine learning

Módulo 1 – Desarrollo
Avanzado en Python



7- SQL aplicado a la Ingeniería de Datos

¿Qué vamos a ver?

01

¿Por qué SQL en Ciencia de Datos?

02

El Modelo Relacional

03

Operaciones fundamentales: SELECT, WHERE, ORDER BY

04

JOINS: construir datasets desde múltiples tablas

05

GROUP BY: feature engineering en el servidor

06

Conectar Python a MySQL con SQLAlchemy

07

Integración SQL + Pandas

08

Flujo profesional completo

09

Consultas parametrizadas, chunking y buenas prácticas

¿Por qué SQL en Ciencia de Datos?

La realidad del trabajo

¿Dónde están los datos?

CSV ✗

Rara vez en producción.

Base de datos relacional ✓

Casi siempre aquí.

El científico de datos debe saber:

Consultar datos con SELECT

Filtrar con WHERE antes de traer

Unir tablas con JOIN

Agregar con GROUP BY

Optimizar y escalar consultas

SQL vs Pandas: herramientas complementarias

No son rivales — tienen roles distintos

SQL hace esto bien

- Filtrar millones de filas en el servidor
- Unir tablas con JOINS
- Agregar datos con GROUP BY
- Aprovechar índices y optimizador
- Consultas reproducibles sobre datos vivos



Pandas hace esto bien

- Análisis exploratorio flexible
- Transformaciones complejas en Python
- Visualización con matplotlib/seaborn
- Feature engineering con lógica Python
- Trabajar con el dataset ya preparado

 Regla de oro: SQL prepara los datos. Pandas los analiza y modela.

El Modelo Relacional

Conceptos que necesitás dominar

Tabla

Conjunto de filas y columnas sobre un tema.

Fila (Registro)

Una observación individual.

Columna (Atributo)

Una variable o característica.

Clave Primaria (PK)

Identificador único de cada fila.

Clave Foránea (FK)

Referencia a la PK de otra tabla.

¿Por qué importa para DS?

Los datos en producción están distribuidos en múltiples tablas relacionadas. Para construir un dataset necesitás unirlos con JOINS. Pandas no conoce esas relaciones — vos tenés que armarlas.

SELECT y WHERE: la base de todo

Operaciones fundamentales

```
-- Columnas específicas (recomendado)
SELECT nombre, ciudad, edad
FROM clientes;

-- Evitar en producción
SELECT * FROM clientes;

-- Filtrar antes de traer
SELECT * FROM ventas
WHERE monto > 10000;

-- Múltiples condiciones
SELECT * FROM clientes
WHERE ciudad = 'Córdoba'
    AND edad >= 25;
```

Operadores WHERE

- = < > <= >= <>
- IN ('A', 'B', 'C')
- BETWEEN valor1 AND valor2
- LIKE '%texto%'
- IS NULL / IS NOT NULL

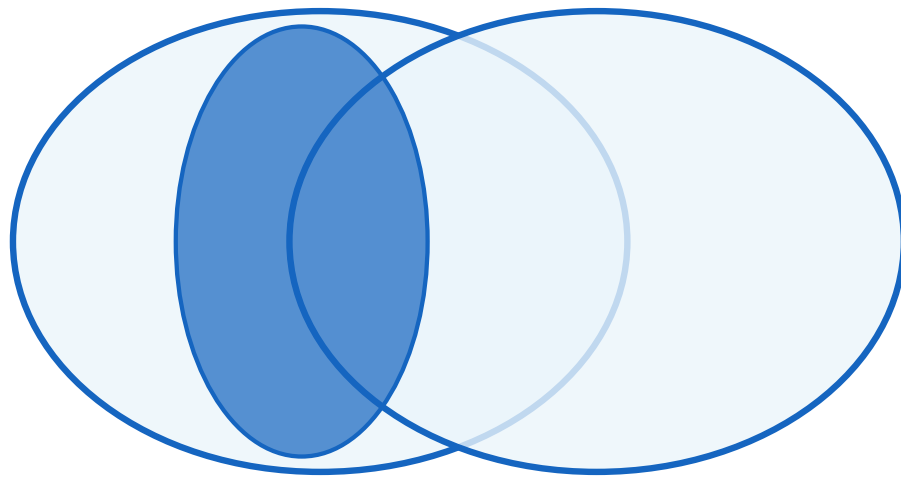
El impacto de WHERE en DS

- ◆ Cada fila que filtrás en SQL es una fila que no cargás en RAM.
- ◆ WHERE reduce el tráfico de red entre el servidor y Python.
- ◆ Siempre filtrá en SQL. Nunca traes todo para filtrar con df.loc.

JOINS: unir tablas para construir datasets

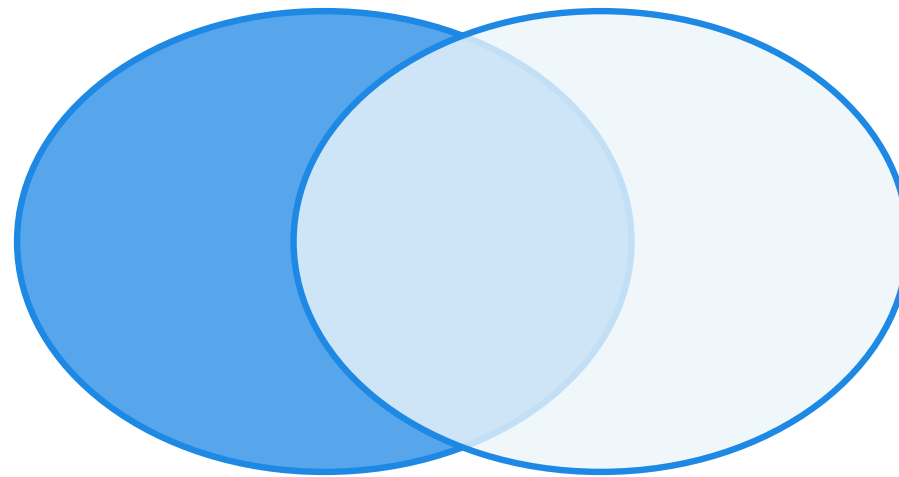
El corazón del modelo relacional

INNER JOIN



Solo coincidencias
en ambas tablas

LEFT JOIN



Todos los de la izquierda
+ coincidencias

```
SELECT c.nombre,  
       v.monto  
FROM clientes c  
LEFT JOIN ventas v  
  ON c.id =  
     v.id_cliente;
```

⚠ **LEFT JOIN es el más importante en DS:**

Con INNER JOIN perdés clientes que nunca compraron. Con LEFT JOIN preservás todos — y eso es fundamental para modelos de churn y retención.

ORDER BY y LIMIT

Ordenar resultados y controlar el volumen de datos

```
-- Ordenar por monto descendente
SELECT nombre, monto
FROM ventas
ORDER BY monto DESC;
-- ASC = ascendente (default)
-- DESC = descendente
```

ORDER BY col ASC

Menor a mayor (default)

ORDER BY col DESC

Mayor a menor

LIMIT N

Traer solo N filas

```
-- Top 10 clientes por gasto
SELECT cliente_id,
       SUM(monto) AS total
FROM ventas
GROUP BY cliente_id
ORDER BY total DESC
LIMIT 10;
```

En DS: ORDER BY + LIMIT es la forma correcta de traer los “top N” a Python. Evita cargar miles de filas para quedarte con 10.

Equivalente Pandas: `df.sort_values('monto', ascending=False).head(10)`

HAVING: filtrar grupos

El complemento indispensable de GROUP BY

WHERE → filtra FILAS

```
-- Filtra antes de agrupar
SELECT ciudad,
       COUNT(*) AS total
FROM clientes
WHERE edad >= 18
GROUP BY ciudad;
↑ filtra FILAS (antes de agrupar)
```

HAVING → filtra GRUPOS

```
-- Filtra grupos resultantes
SELECT ciudad,
       COUNT(*) AS total
FROM clientes
GROUP BY ciudad
HAVING total > 5;
↑ filtra GRUPOS (después de agrupar)
```

Orden de ejecución SQL: FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT

Caso DS: “Dame los clientes con más de 5 compras” requiere HAVING. WHERE no puede filtrar por COUNT() porque COUNT se calcula después de agrupar.

Equivalente Pandas: `df.groupby('ciudad').filter(lambda x: len(x) > 5)`

GROUP BY: feature engineering en el servidor

Transformar transacciones en features

```
-- Dataset para modelo de segmentación
SELECT
  c.id,
  c.ciudad,
  COUNT(v.id)           AS frecuencia,
  SUM(v.monto)          AS gasto_total,
  AVG(v.monto)          AS
ticket_promedio,
  MAX(v.fecha)          AS ultima_compra,
  DATEDIFF(NOW(), MAX(v.fecha)) AS recencia
FROM clientes c
LEFT JOIN ventas v ON c.id = v.id_cliente
GROUP BY c.id, c.ciudad
HAVING gasto_total > 0
ORDER BY gasto_total DESC;
```

¿Qué construye este query?

frecuencia

Frequency del modelo RFM

gasto_total

Monetary del modelo RFM

recencia

Recency del modelo RFM

ticket_promedi

o

Feature de comportamiento

SQL no entrena modelos — SQL construye el dataset que va al modelo. Ese es su rol.

JOIN en SQL = merge() en Pandas

Misma lógica, distinta sintaxis

SQL	Pandas	Resultado
INNER JOIN	how='inner'	Solo coincidencias
LEFT JOIN	how='left'	Todos de la izquierda
RIGHT JOIN	how='right'	Todos de la derecha
FULL OUTER JOIN	how='outer'	Todo, NaN si no hay

SQL	Pandas
<pre>SELECT c.nombre, v.monto FROM clientes c LEFT JOIN ventas v ON c.id = v.cliente_id;</pre>	<pre>clientes.merge(ventas, left_on='id', right_on='cliente_id', how='left')</pre>

Regla: Si el JOIN se puede hacer en SQL antes de traer los datos, hazlo en SQL. Usar merge() cuando los datos ya están en Python.

Los 4 Tipos de JOIN

◆ Sintaxis y Parámetros

```
import pandas as pd

# (1) INNER JOIN
inner = pd.merge(df_izq, df_der, on='id', how='inner')

# (2) LEFT JOIN
left = pd.merge(df_izq, df_der, on='id', how='left')

# (3) RIGHT JOIN
right = pd.merge(df_izq, df_der, on='id', how='right')

# (4) OUTER JOIN (FULL)
outer = pd.merge(df_izq, df_der, on='id', how='outer')
```

- ✓ **on:** Columna(s) clave para unir (id_prod).
- ✓ **how:** El tipo de JOIN (inner, left, right, outer).

Left Join



Todo el DataFrame izquierdo + coincidencias del derecho.

Outer Join



Unión total: mantiene todas las filas de ambos lados.

Inner Join



Solo filas con coincidencias en AMBAS tablas.

Right Join



Todo el DataFrame derecho + coincidencias del izquierdo.

Validación y Seguimiento

```
# Uso profesional con validación df_final =  
pd.merge( df1, df2, on='id', how='left',  
validate="one_to_many", indicator=True ) #  
Ver origen de cada fila  
print(df_final['_merge'].value_counts())
```

Dato clave: Donde no hay coincidencia, Pandas rellena automáticamente con valores **NaN** (Not a Number).

Tips de Seguridad

- **validate:** Evita errores de duplicidad inesperados.
- **indicator:** Agrega la columna `_merge` para saber de dónde vino el dato.
- **suffixes:** Controla los nombres si hay columnas repetidas.

Conectar Python a MySQL

SQLAlchemy es el puente recomendado

```
from sqlalchemy import create_engine
import pandas as pd

# Formato: dialect+driver://user:pass@host/db
engine = create_engine(

'mysql+pymysql://usuario:clave@localhost/empresa'
)

# Leer directo a DataFrame
df = pd.read_sql('SELECT * FROM clientes', engine)
print(df.head())
```

Pasos para conectar

- pip install sqlalchemy pymysql pandas
- Crear el engine con la cadena de conexión
- Usar pd.read_sql(query, engine)
- engine.dispose() al terminar

Python

SQLAlchemy
Engine

Servidor
MySQL

Base de
Datos

Integración SQL + Pandas

read_sql() y to_sql(): los dos sentidos del flujo

```
# SQL → Pandas (leer)
df = pd.read_sql('SELECT nombre, monto FROM
ventas', engine)
```

```
# Pandas → SQL (escribir)
df_resumen.to_sql(
    'clientes_resumen', engine,
    if_exists='replace', index=False
)
```

if_exists opciones

'fail'

Error si la tabla existe.

'replace'

Borra y recrea la tabla.

'append'

Agrega filas al final.

Usá siempre index=False en to_sql() para no guardar el índice numérico de Pandas como columna en la BD.

Flujo profesional completo

El pipeline estándar en la industria

1

**Leer datos
crudos**

CSV, API
o BD fuente

2

**Limpiar
en Pandas**

Tipos, nulos,
normalizar

3

**Persistir
en SQL**

df.to_sql()
to MySQL

4

**Consultar
con SQL**

JOIN + GROUP BY
+ WHERE

5

**Analizar
en Pandas**

Features +
Modelo ML

La clave del flujo: SQL hace el trabajo pesado antes de que Pandas lo vea. Nunca al revés.

Consultas parametrizadas

La forma correcta y segura de queries dinámicas

```
# ❌ MAL: concatenación directa
query = "SELECT * FROM clientes "
      + "WHERE ciudad = '" + ciudad + "'"
```

```
# ✅ BIEN: parámetro con text()
from sqlalchemy import text

query = text('SELECT * FROM clientes
             WHERE ciudad = :c AND edad > :e')
df = pd.read_sql(query, conn,
                 params={'c': 'Córdoba', 'e': 25})
```

¿Por qué importa?

- Previene inyección SQL
- Reutilizable con distintos valores
- Mejor rendimiento (plan cacheado)
- Código más legible y mantenible

Función reutilizable

- `def consultar(sql, params=None):`
- `with engine.connect() as conn:`
- `return pd.read_sql(`
- `text(sql), conn,`
- `params=params or {})`

Chunking y tipos de datos

Escalar y evitar errores silenciosos

chunksize: tablas grandes sin colapsar RAM

- `chunks = pd.read_sql(query, engine, chunksize=50000)`
- `for chunk in chunks:`
- `total += chunk['monto'].sum()`
- # Procesa en bloques, no carga todo en RAM

¿Cuándo usarlo?

- Tabla con más de 500k filas
- Pandas arroja `MemoryError`
- El kernel de Jupyter se muere

Maapeo SQL → Pandas

INT (sin NULLs)	int64
INT (con NULLs)	float64 ⚠
FLOAT / DOUBLE	float64
VARCHAR / TEXT	object
DATE	object → to_datetime()
DATETIME	datetime64[ns]

Siempre verificá `df.dtypes` después de `read_sql()`. Los tipos no siempre llegan como esperás.

Errores comunes que hay que evitar

Anti-patrones que cuestan tiempo y recursos

✗ **SELECT * + filtrar en Pandas**

Traer millones de filas para luego hacer `df[df['x']=='y']`. Eso quema RAM innecesariamente.

✗ **Ignorar LEFT JOIN**

Usar solo INNER JOIN hace que pierdas clientes sin actividad — distorsiona el análisis.

✗ **Concatenar strings en queries**

`f"WHERE ciudad = '{ciudad}'"` es una vulnerabilidad de inyección SQL. Usá `params={}`.

✗ **No cerrar conexiones**

Olvidarse de `engine.dispose()` agota el pool del servidor en scripts recurrentes.

✗ **Hardcodear credenciales**

Contraseñas en el código → leak en el primer commit a GitHub.

Buenas prácticas consolidadas

Lo que hace la diferencia en producción

✓ Filtrá en SQL primero

WHERE y LIMIT antes de traer datos a Python.

✓ Manejá errores

try/except con OperationalError y ProgrammingError.

✓ Nombrá las columnas

SELECT id, nombre, monto — nunca SELECT *.

✓ Cerrá conexiones

engine.dispose() o with engine.connect() as conn.

✓ Usá params={}

Nunca concatenes strings para queries dinámicas.

✓ Variables de entorno

.env + python-dotenv, nunca credenciales en el código.